Software Process Management

Lecture 3

Recap

- We discussed an overview of software, and their development
- We discussed about the origin of software problems
  - Requirements and Requirement Engineering
- Now let's get into some serious stuff!
  - Explore the challenges of modern software development
  - That we define why do we need a process
  - And start understanding the process and process models
    - Along with our expert presentations on process models

Software Project Challenges

- Product quality poor – but what should we change to fix this?
- e.g., when do we introduce most bugs – and when should we find them?

- Deadlines missed – but where do we lose all that time?
- e.g., do we lose most time during requirements negotiation, or during testing?

- Budget overruns – but what are the most expensive activities?
- e.g., are we spending most effort on coding or on rework?
- Overhead – why do we spend so much effort on non-productive activities?
- e.g., how much time is consumed by collecting information?
- New employees – how should they learn how we work?
- e.g., through trial & error or by some more efficient means?

To address these challenges…

- What information do we need to answer these questions?
- Which activities are performed in which sequence?
- Which are the inputs and outputs of these activities?
- Who is involved in a specific activity?
- How long does it take to complete an activity?
- Are specific people bottlenecks?
- How do the activities contribute to higher-level goals?
- …

- How can we gather the required information?
- We need a precise and concise model of the process that holds the required information
- Again…
- *If you don't know where you are, a map won't help!*

Team-oriented Software Development

- Many activities are not performed by individuals, but are shared among different developers working together smoothly.

- In large projects, a multitude of activities can be performed in parallel.
- This requires good coordination so that the results of these tasks fit together in a planned way.

Team-oriented Software Development

- There are many relationships between activities.
- Documents or code, for instance, can be exchanged between activities or may be used jointly by different activities.
- In addition, temporal dependencies may exist between activities.

- – Many activities need to be synchronized so that they contribute to overall project goals.
- In systems engineering, for instance, software engineering processes often need to be synchronized with processes from mechanical electronic and electrical engineering (what else?)

Team-oriented Software Development

- As software development is a largely human-based activity, building teams is an important issue.
- Teamwork involves, for instance, selecting a team, harmonizing the contributions of individual members, integrating different skills and interests, and solving conflicts.
- Clear responsibilities can help to overcome problems with team development.
- Managing human-based processes requires great leadership skills. One of the main tasks is to motivate people to contribute to common goals.

Team-oriented Software Development

- Besides product requirements, project managers have to consider process requirements when performing projects and leading teams.
- Examples of process requirements include adherence to process standards or required productivity.

- Therefore,
- Coping with these difficulties requires a description of what process is being followed!

Global Software Development

- Software and systems development is being increasingly performed concurrently in different countries with many customer–supplier relationships along the development chain.
- Outsourcing, offshoring, and near-shoring are aggravating this trend.
- Global software development is close to becoming the norm.



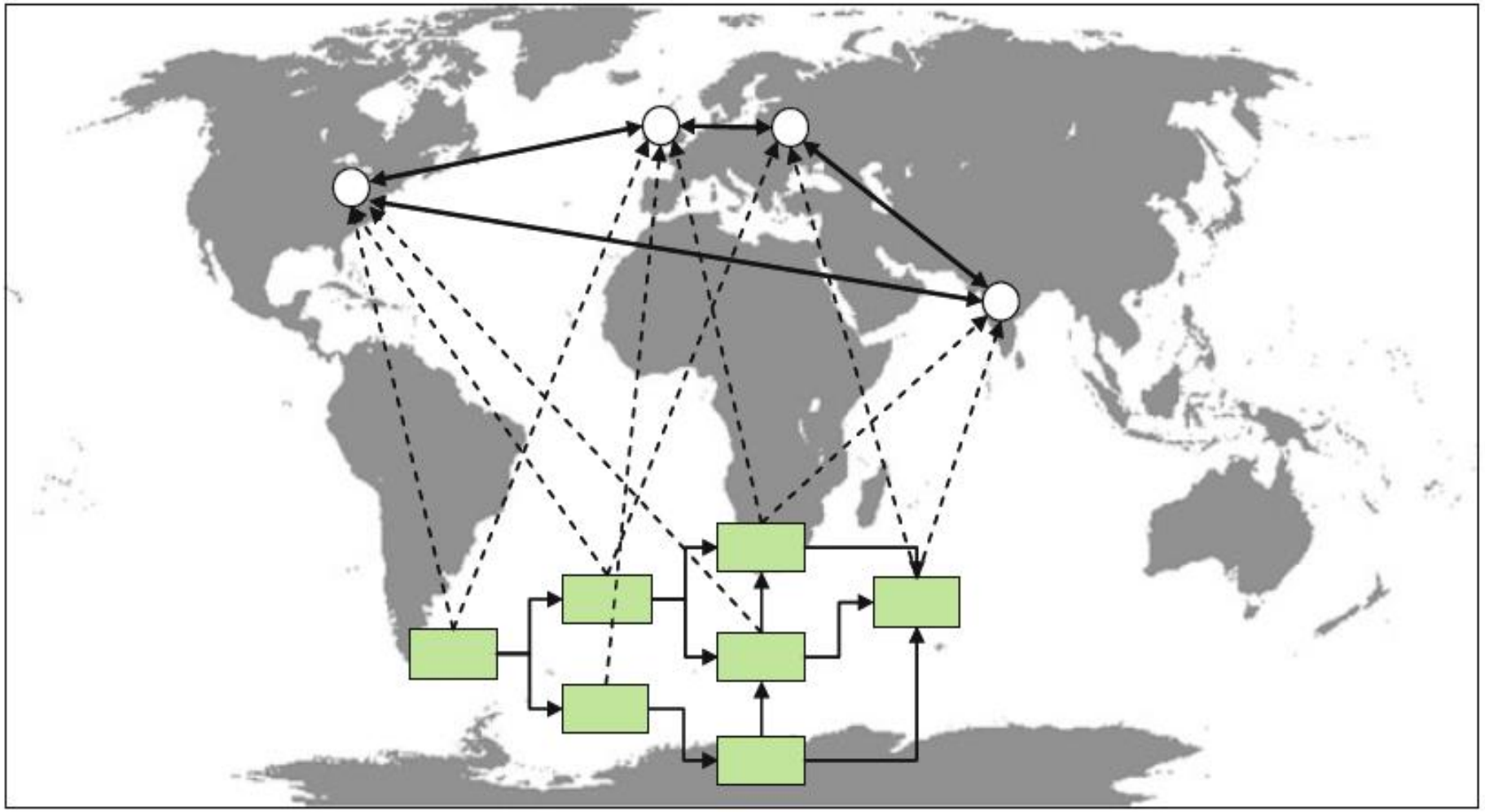
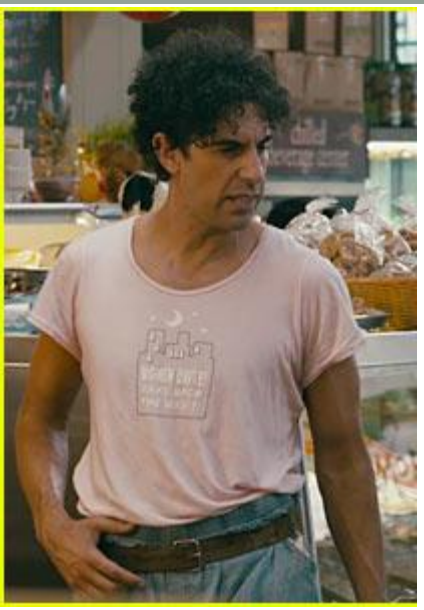


Fig. 1.2 Distribution of software engineering tasks to different sites










PLAYERS


GOLDEN GLOBE AWARDS
NBC

- It is inherently more difficult to coordinate projects where teams are physically distributed than projects with co-located teams.

- Supporting distributed development requires well-understood process interfaces and process synchronization!

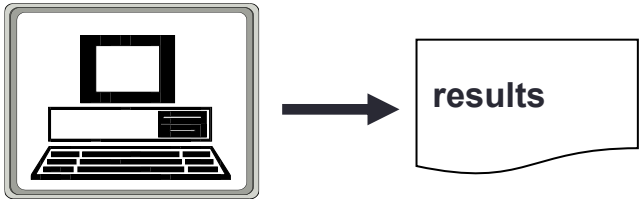
Example – Telco Domain

- Telecommunication systems
- Approximately 400 developers in 5 locations
- Between 2,500 and 3,000 modules (Pascal, Assembler, C/C++)
- Basic service on PABX and extended service on PC platform
- Release cycles between 1 and 2 years
- More than two dozen different versions installed at different customer sites
- Different disciplines such as mechanical electronic and electrical engineering are involved

- Coping with this context requires a clear description of which activities are performed in which sequences on which development location!

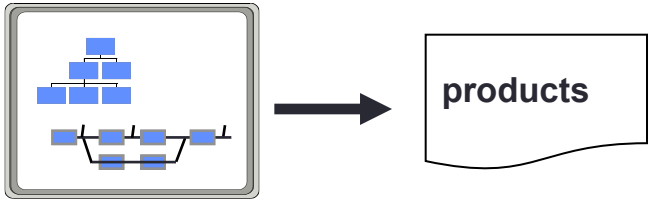
Product Quality Vs Process Quality

- When software provides incorrect results ....



... we do not modify the results, but the software that produces them.☺

- When a process provides incorrect products ...



... then common practice is to modify the products, instead of the process. ☹

- Product is precious



- Yet often difficult to see till the end…

High **product quality** routinely results from high **process quality**.“

If no SP management

- If no adequate process management is established, can cause problems such as
- – Unnecessary rework
- – Deviations from plan are detected too late
- – Confusion regarding roles and responsibilities
- – Documents cannot be found when needed because they are not associated with process steps
- – Variations in process execution
- – Permanently incomplete and inconsistent process documentation

!SP management Cont.

- – Deferred certification because appropriate process documentation cannot be produced promptly
- – Performance inefficiency (due to “unfit for purpose” processes)
- – Uncertain execution and dependence on individual efforts (due to vague and incomplete process descriptions)
- – Inefficient division of work and double work (due to poorly defined interfaces)

Software Process overview

- Often, software processes are also called “software development processes”.
- It is recommended using the term software process because
- (a) many processes are not directly related to development (such as maintenance processes), and
- (b) software processes are also relevant in areas where not only software is produced (e.g., when building software-based systems).

Characteristics of software processes

- They are enacted in the real world
- They usually transform one or more input products into one or more output products by consuming further products (e.g., guidelines)
- They can be performed by humans (“enactment“) or machines (“execution“) or both together
- They can be refined by sub-processes, each of which can also be refined

Project vs Process

- A project *is a unique endeavour, which is limited by a start date and an end date and should achieve a goal*

- *It should be recognized that permanent or routine activities are not projects.*
  - E.g. Generic activities against specific tasks

Project Phases

- A project phase (or *phase*) is a collection of logically separated project activities, usually culminating in the completion of a major deliverable or the achievement of a major milestone.

- Relate this with the notion of *projects*

Characteristics of project phases

- Phases are mainly completed sequentially, but can overlap in some project situations
- Phases can be subdivided into sub-phases
- Unlike a process, a phase is always defined by a start date and an end date. If this period is finished, the phase is finished. Typically, processes can be activated multiple times
- Typical examples of phases are the elaboration phase, the construction phase, or the transition phase.
- Phases are usually used when looking at a project from a management perspective

Project Phase vs Process

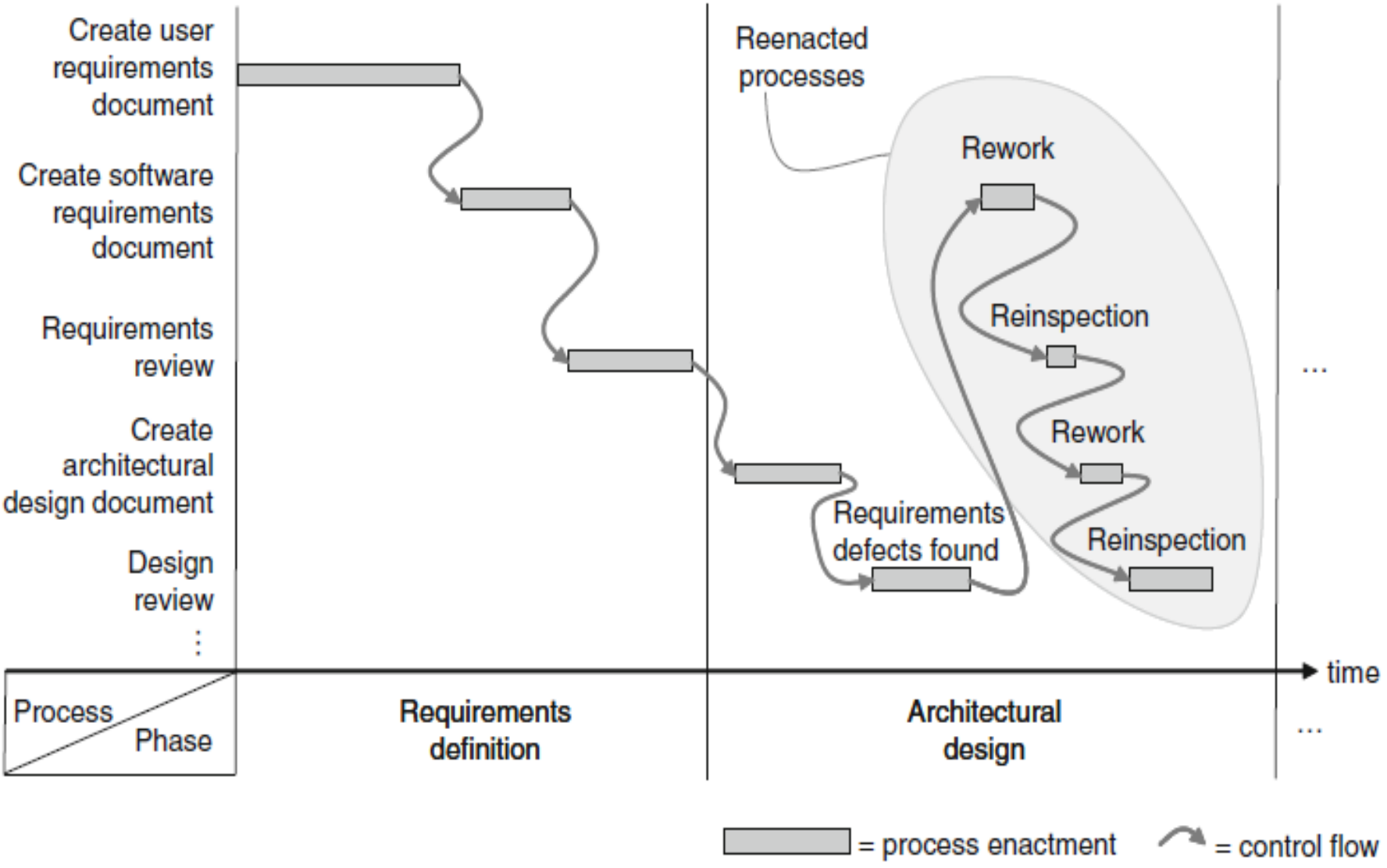
- Every project has
  - An Engineering View
  - A Management View

- Management often needs a period based view on activities because investors, investments, dividends, revenue calculations, and financial plans are typically time based

Project Phase vs Process

- Engineers usually need a product-based view on activities
- i.e., a view on activities that describe in a goal-oriented way the steps needed to create, maintain, or operate a product or service.

- Project phase can only be performed once!


Create user requirements document
Create software requirements document
Requirements review
Create architectural design document
Design review
Reenacted processes
Rework
Reinspection
Rework
Reinspection
Requirements defects found
...
time
Process
Phase
Requirements definition
Architectural design
= process enactment
= control flow

Software Process Models

- 1) What are they?
- 2) Why do we need them?
- 3) Examples?

Terminology

- A Model:
- An abstract and simplifying representation of an object or phenomenon of the real world (usually)
- A software process model:
- A model of a software process! ☺

Characteristics of Models

- They describe only those aspects of the object or phenomenon that are (believed to be) relevant for the understanding and intended usage of the model
- They encapsulate experience and allow for an explicit representation of experience
- They can be created for different purposes such as planning, control, or prediction
- They have their own lifecycle, i.e., they can be specified, built, implemented, analysed, used, assessed, evolved, or rejected

We must remember that

- “Essentially, all models are wrong, but some are useful,” attributed to the statistician George Edward Pelham Box, highlights that a model only represents a limited number of real-world aspects and details.

- Reflect ADTs and OO*X* : {x|x= design, development, programming, analysis, *please continue*…}

- The challenge lies in capturing sufficient real-world aspects and details in a model so that the model can be used for its purpose.

Software Process Models

- A software process model is a description of a software process.
- Process models are often used as a means for problem solving.
- The specification of the enactment of a software process by a process model is comparable to the specification of baking a cake using a recipe.
- Process models can be represented by using different notations (e.g., graphical, natural language, machine-readable notations).
- A process model can describe a process on different levels of abstraction (e.g., lifecycle process level, engineering process level, atomic step level).

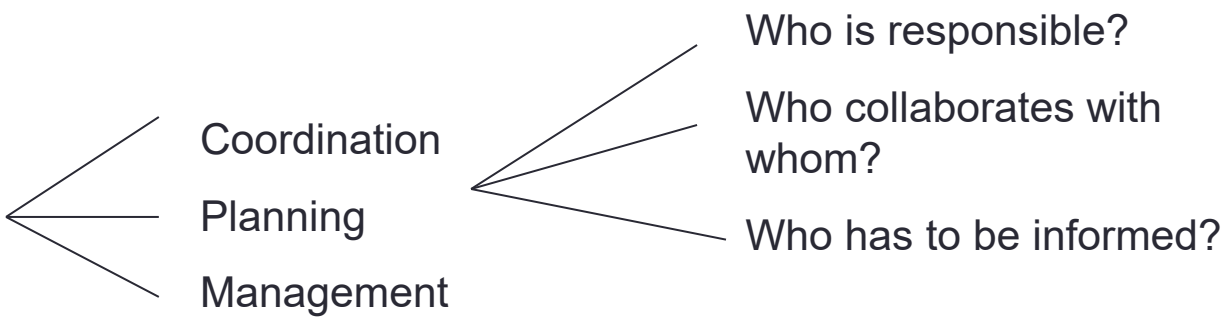
Process Modelling Goals

- To enable effective understanding and communication
  - At one development site (developers, teams, ...)
  - Between development sites (distributed development, outsourcing, contractor-supplier relations, ...)
- To improve software development activities
  - Improving real processes requires measurement and measurement requires defined processes
  - Evolving processes
- To support process and project management
  - Transparency, tracking, ...
- To guide the developers
  - incorporating new employees
  - Reuse of process knowledge and support process automation

Expected benefits

- Following benefits are expected from systematic process modelling
- Better transparency of software engineering activities

- - External
- - Internal



Cont.

- Reduced complexity of large development efforts

- The ability to perform process measurement (i.e., process models that are used in practice are a prerequisite for process measurement and, in consequence, for process improvement)

Cont.

- Predictability with respect to the process characteristics and the characteristics of the results is only achievable with explicit models

- (i.e., enabling predictability for characteristics such as
  - consumed effort,
  - completion date, or
  - reliability of a produced software component
- requires the existence of explicit process models;
  - although this is not enough and other models are needed, too)

SPM Cont.

- Compared to other engineering disciplines, software engineering software process modelling is a young discipline.
- A mature or standardized terminology does not exist.
- Besides the newness of the domain, this is mainly caused by the parallel emergence of different process notations and influences from different other domains such as
  - Business process modelling
  - Programming notations.

SPM and context

- The characteristics of the environment, the context, determines which process models are the most appropriate for the given situation
  - Context factors
  - Team experience
  - Application domain
  - Available budget
  - Non-functional quality requirements for the system
  - Reliability
  - Accuracy
  - Security
  - Cost
- Process models need to be tailored for the context and improvement needs to be context-specific

Best method fallacy

- Cool Method (Agile, structured, OO, SPI, Lean, Whatever)



- We should know this
- Malouin, J. L. and M. Landry (1983). The Miracle of Universal Methods in Systems Design. Journal of Applied Systems Analysis 10:47-62.